TOPIC 2 · WEB APPLICATION SECURITY — PART 1 OF 3

Web Application Security — Part 1 of 3

How web apps break, how to map them, and the two access-control failures that top the list

Topic 2 · Web Application Security (15h, delivered across 3 parts) · LO1 information gathering · LO2 evaluate tools & techniques

What we'll cover

1. How web apps break — the client/server trust boundary
2. Mapping the application — turning recon into a target map
3. Burp Suite — the tester's core workflow
4. The OWASP Top 10 (2021) — the map for this web block
5. A01: Broken Access Control — IDOR & forced browsing
6. A07: Identification & Authentication Failures
7. Mitigations & key takeaways

PART A

How web apps break

The one trust boundary that every web vulnerability lives on

The web is the biggest attack surface you'll test

Almost every organisation exposes a web application to the entire internet — it is the door that is always open.

- A network service can be firewalled off; a public web app must, by definition, accept traffic from anyone, anywhere.
- It is also the richest surface: authentication, business logic, file handling, payments and APIs all sit behind one URL.
- This is why Topic 2 gets 15 hours across three parts — more than any other block — and why the prescribed text is a web-testing book.
- The three parts map onto the OWASP Top 10: access control and authentication today, the injection family next, then client-side and logic.

Analogy: *A shop can lock the loading bay and the staff door, but the front entrance has to stay open to customers — that is the web app.*

ANATOMY

A web application is a stack, not a page

What looks like one website is really four layers, and a tester probes each one differently.

- Browser (client): renders HTML/CSS, runs JavaScript — fully under the attacker's control once it leaves the server.
- Transport: HTTP over TLS — the requests and responses a tester intercepts and tampers with.
- Server + application: the web server (nginx, Apache) and the app code (PHP, Node, Java) that makes the real decisions.
- Data tier: the database, file store and downstream APIs the app talks to — the prize behind the app.

Analogy: *A restaurant: the dining room (browser) takes your order, but the kitchen (server) actually decides what gets cooked.*

THE CORE IDEA

The client/server trust boundary

Everything on the attacker's side of this line can be forged. Everything on the server's side is where security must actually live.

- Anything the browser sends — URL, parameters, headers, cookies, JSON body — is fully attacker-controlled and can be changed to any value.
- The server is the only place that can be trusted to enforce a rule, because it is the only place the attacker cannot rewrite.
- Every web vulnerability in this course is, at root, the server trusting something it received from the client.
- Learn to see the boundary and you can predict where the bugs are before you find them.

Analogy: *A self-service checkout shows a price, but the till must re-scan the barcode — trusting the price the shopper typed is how a laptop leaves for £1.*

THE GOLDEN RULE

Never trust input from the client

Client-side validation is a convenience for honest users; it is never a security control.

- JavaScript that checks a form, a disabled button, a hidden field, a maxlength — all of it runs in territory the attacker owns.
- The tester's reflex: whatever the browser enforces, remove it and send the request directly with Burp or curl.
- A 'price' field hidden in a form, a role sent as role=user, a maxlength on a field — each is an invitation to change it.
- Rule for the whole block: if a control only exists in the browser, it does not exist at all.

Analogy: *A bouncer who believes whatever birth-year a guest writes on a napkin — the check has to happen at the door, not on the napkin.*

ATTACKER'S VIEW

An HTTP request, read as a tester

Year 2 covered what a request is. A tester reads every line as something to tamper with:

```
POST /api/orders HTTP/1.1          <- method + path: try PUT/DELETE too
Host: shop.target.com              <- routing: header-injection target
Cookie: session=8f2a...; role=user <- session + a role a tester will flip
Authorization: Bearer eyJhbGci...  <- a JWT (decode + attack it, Part F)
X-Forwarded-For: 127.0.0.1         <- spoofable 'trusted' header
Content-Type: application/json

{ "item": 55, "qty": 1, "price": 999 }   <- change qty to -1, price to 1
```

Every element above is attacker-controlled. The test is always: what does the server do when this value is a lie?

ATTACKER'S VIEW

An HTTP response, read as a tester

The response leaks intent. A tester reads it for what it reveals and what it forgot to set:

```
HTTP/1.1 302 Found                    <- redirect may hide a gated page (Part E)
Set-Cookie: session=8f2a...           <- missing HttpOnly/Secure/SameSite? (Part F)
Server: Apache/2.4.49                 <- version -> map to a known CVE (recon)
X-Powered-By: PHP/7.2.1               <- tech + version leak
WWW-Authenticate: ...                 <- auth scheme in use
  (missing) Content-Security-Policy <- no CSP -> XSS is easier (Part 3)
  (missing) X-Frame-Options          <- framable -> clickjacking
```

A 302 does not mean 'access denied' — the gated content is often in the response body before the redirect fires.

How the app remembers who you are

HTTP is stateless, so the app carries identity in something the client holds — and whatever the client holds, the client can tamper with.

- Cookies: the classic session carrier — a name=value pair the browser returns on every request.
- Bearer tokens / JWTs: sent in the Authorization header, common in APIs and single-page apps (attacked in Part F).
- Hidden form fields and URL parameters: state pushed into the page itself — trivially editable.
- Local/session storage: tokens kept in the browser by JavaScript — readable by any script that runs on the page (matters for XSS, Part 3).

Analogy: *A coat-check ticket: the cookie is the ticket, and whoever holds the ticket collects the coat — the cloakroom never re-checks your face.*

THE MINDSET

Every input is a question the server answers

1. **Enumerate every entry point** URL params, form fields, headers, cookies, JSON keys, file uploads, path segments

2. **Each one is a question** the server answers it — sometimes it answers questions you never intended to ask

3. **Vary the type, not just the value** send a string where a number is expected, an array where a string is, a negative where a count is

4. **Watch the response delta** different error, different timing, different length — each is a signal something changed inside

REFERENCE TABLE

Methods & status codes a tester actually cares about

Beyond GET/POST/200/404 from Year 2 — the ones that carry security meaning:

ITEM	MEANING TO A TESTER
PUT/DELETE	State-changing verbs — often unprotected vs GET/POST
PATCH	Partial update — mass-assignment surface (Part E)
OPTIONS	Reveals allowed methods + CORS policy
401	Unauthenticated — you have no valid identity
403	Authenticated but forbidden — authz boundary (probe it)
302	Redirect — may hide gated content in the body
405	Method not allowed — try a different verb
500	Server error — stack trace / injection signal

401 vs 403 is the exam-relevant pair: 401 = 'no valid identity', 403 = 'identity known, but not permitted'. 403 is where access-control bugs hide.

PART B

Mapping the application

Turning last week's recon into a room-by-room map of one app

Mapping is recon aimed at one application

Reconnaissance produced a prioritised list of hosts. Mapping takes the top host and enumerates every page, parameter and function inside it.

- The goal is a complete inventory of entry points: URLs, parameters, forms, APIs, upload fields, and the auth mechanism.
- You cannot test what you have not found — an un-mapped admin endpoint is an un-tested one.
- This is OWASP WSTG-INFO (Information Gathering), the checklist that opens every web engagement.
- Output: an annotated site map you drive the rest of the block from — the direct input to A01 and A07 testing.

Analogy: *Reconnaissance gave the satellite photo of the street; this is the floor plan, drawn room by room from the inside.*

Spidering and browsing the app

First pass: let the tools and your own clicking discover the linked content the app admits to having.

- Automated crawl: Burp's crawler or OWASP ZAP's spider follows every link and form to build a site map.
- Manual walk-through with the proxy on: every page you visit is captured, so browsing the app IS the mapping.
- Crawlers miss what is not linked — content reachable only by knowing the URL (that is the next slide's job).
- Log in and crawl again: authenticated content is a different, larger map than the anonymous one.

Analogy: *Following every corridor that has a sign — thorough, but it never finds the unmarked door.*

TYPE THESE TONIGHT

Forced browsing — finding the unlinked content

Brute-force paths and files that no link points to (run against the lab, DVWA :8080 / Juice Shop :3000):

```
# ffuf — fast content discovery (ffuf.me / github.com/ffuf/ffuf)
$ ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt \
       -u http://localhost:8080/FUZZ -mc 200,204,301,302,401,403

# gobuster — same job, add file extensions
$ gobuster dir -u http://localhost:8080 \
       -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak

# feroxbuster — recursive by default
$ feroxbuster -u http://localhost:3000 -w common.txt
```

SecLists (github.com/danielmiessler/SecLists) is the wordlist collection. A 403 is a find, not a failure — something is there, and access control is guarding it (Part E).

Parameter discovery

Endpoints often accept parameters that appear in no form and no link — undocumented inputs are untested inputs.

- A page may quietly honour ?debug=true, ?admin=1, ?id=, or a JSON key the UI never sends.
- Arjun (github.com/s0md3v/Arjun) brute-forces parameter names against an endpoint and reports which change the response.
- Burp's Param Miner extension (BApp store) does the same for query, body and header parameters.
- Hidden parameters are a direct route to mass assignment and access-control bugs (Part E).

Analogy: *Trying words in a lock that has no keyhole showing — some of them still turn something inside.*

KNOW THE STACK

Fingerprinting the technology

Identify the server, framework and versions — each one maps to a body of known vulnerabilities.

- Signals: Server and X-Powered-By headers, cookie names (PHPSESSID, JSESSIONID, connect.sid), error-page style, file extensions.
- Tools: whatweb (whatweb http://localhost:3000), Wappalyzer browser extension, and Burp's passive fingerprinting.
- A framework name plus a version number is a search query: 'Struts 2.5.10 CVE', 'Rails 5.2 vulnerability'.
- Old components are A06 on the Top 10 (Vulnerable & Outdated Components) — a finding in themselves.

Analogy: *Reading the maker's plate on a machine — model and year tell you exactly which recalls apply.*

Reading the app's own JavaScript

Modern apps ship their routing and API map to the browser — the front-end code is a gift to the tester.

- JavaScript bundles frequently list API endpoints, hidden routes, feature flags and occasionally hard-coded secrets.
- Grep the JS for /api/, fetch(, axios, URLs and keywords like token, key, admin, secret.
- Tools: LinkFinder and the browser dev-tools Sources tab; Burp collects every .js file automatically in the site map.
- A single-page app's whole route table is usually one JavaScript file away.

Analogy: *The kitchen accidentally handing you the full recipe book with the menu — every dish, including the ones not on display.*

THE DELIVERABLE

The target profile you build before attacking

1. **Entry points** every URL, parameter, form, API route, upload and header the app accepts

2. **Authentication mechanism** how login works, what a session token looks like, where it is carried

3. **Roles & functions** anonymous vs user vs admin — and which functions each is supposed to reach

4. **Technology & versions** server, framework, libraries — each a lead for known-CVE research

PART C

Burp Suite — the core tool

The intercepting proxy you will live inside for the whole web block

WHAT IT IS

Burp Suite: an intercepting proxy

Burp sits between the browser and the server so every request pauses on the tester's desk before it is sent.

- It intercepts, inspects and lets you modify any request or response in flight — the client-side controls from Part A simply stop mattering.
- Burp Suite Community Edition is free (portswigger.net/burp); Professional adds the active scanner and unthrottled Intruder.
- OWASP ZAP (zaproxy.org) is the free, open-source alternative with the same proxy-and-tamper core workflow.
- The whole point: it moves the fight from the browser (attacker-owned but limited) to the raw HTTP the server actually reads.

Analogy: *A wiretap you are authorised to run — every call pauses at your console before it connects, and you can rewrite it.*

TYPE THESE TONIGHT

Point the browser through Burp

One-time setup so the browser's traffic flows through Burp (do this in the lab):

```
1. Start Burp -> Proxy tab -> confirm listener on 127.0.0.1:8080
   (Burp's own port; the DVWA target is a separate :8080 container)
2. Point the browser proxy at 127.0.0.1:8080
   - easiest: Burp's built-in browser (Proxy -> Open browser)
   - or the FoxyProxy extension with a Burp profile
3. Install Burp's CA certificate to intercept HTTPS:
   browse to  http://burp  -> CA Certificate -> import into the browser
4. Proxy -> Intercept is on -> browse the app -> requests now pause in Burp
```

Without the CA cert, HTTPS pages show certificate errors. Burp's embedded browser has the cert pre-installed — the fastest path in a lab.

PROXY TAB

Proxy — intercept and HTTP history

The heart of Burp: pause traffic, read it, edit it, and keep a searchable log of everything that passed through.

- Intercept: hold a request, change any part (parameter, header, cookie, body), then forward it to the server.
- HTTP history: a complete, filterable record of every request/response — the map of what the app actually did.
- This is where client-side validation dies: edit the request after the browser's JavaScript has already run.
- Right-click any captured request to send it to Repeater, Intruder, Comparer or Decoder — the hub of the workflow.

Analogy: *A photographer's contact sheet: every shot the app took is here, and you can pull any one out to work on.*

TARGET TAB

Target — site map and scope

Burp assembles everything it has seen into a tree of the application, and lets you fence testing to what is authorised.

- The site map grows automatically as you browse and crawl — the visual form of the Part B mapping.
- Scope: add only the in-scope hosts, then 'show only in-scope' — Burp stops logging and attacking anything else.
- Setting scope is a Rules-of-Engagement control (from the scoping phase): it enforces the out-of-scope wall inside the tool.
- Grey-out shows untested items; it doubles as a coverage checklist for the engagement.

Analogy: *A site plan with the permitted work area outlined in red tape — everything outside it is off-limits by construction.*

Repeater — the manual tampering workhorse

Send one request, edit any part, resend, and read the response — the single most-used tool for finding bugs by hand.

- The core loop of manual testing: change one thing, resend, compare the response — the Part A input/response loop, mechanised.
- Every access-control test in Part E is a Repeater action: change an id, drop a cookie, swap a token, resend.
- Requests sit in numbered tabs so you can keep a whole test case per tab and return to it.
- No automation, no noise — precise, surgical, one request at a time. This is where craftsmanship lives.

Analogy: *A photocopier for requests: change one word on the original, run another copy, hold the two up side by side.*

Intruder — automating the same request

When one request must be sent hundreds of times with a changing value, Intruder marks the positions and fires the payloads.

- Mark payload positions (the id, the password, the token), choose a payload list, and Burp iterates automatically.
- Uses: fuzzing a parameter, enumerating IDs for IDOR, brute-forcing a login, testing every value of a field.
- Sort results by status code and response length — an outlier length is usually the interesting response.
- Community Edition throttles Intruder heavily; it still teaches the concept, and Pro removes the limit.

Analogy: *A robotic arm that retypes the same request a thousand times, changing one word each pass, and logs every answer.*

REFERENCE TABLE

Intruder's four attack types

How Intruder combines payload positions with payload sets — pick by what you are testing:

TYPE	POSITIONS	PAYLOADS	USE FOR
Sniper	1 at a time	1 set	Fuzz one field; enumerate IDs
Battering ram	all at once	1 set (same)	Same value in every position
Pitchfork	many	1 set each	Paired lists (user+matching id)
Cluster bomb	many	all combos	Username x password brute-force

Cluster bomb tries every combination of the sets — powerful, but the request count is the product of the list sizes, so it grows fast.

COMPARER + DECODER

Comparer and Decoder — read the delta, unwrap the data

Two small tools that close the loop: one shows exactly how two responses differ, the other decodes the encodings apps hide data in.

- Comparer: word- or byte-level diff of two responses — proves whether tampering actually changed the server's answer.
- Decoder: encode/decode Base64, URL, HTML, hex; useful for reading a token or crafting an encoded payload.
- The pair supports the Part A loop: Decoder to understand a value, tamper it, Comparer to confirm the effect.
- A JWT is Base64url — Decoder reveals its contents in seconds before you attack it in Part F.

Analogy: *Comparer is spot-the-difference between two nearly identical pictures; Decoder is the cipher wheel that turns gibberish into text.*

A full Burp pass on a login

The tools chained into one realistic sequence against the lab login:

```
1. Proxy ON  -> submit the login form -> request pauses in Intercept
2. Read it: POST /login  body user=admin&pass=admin  (client checks bypassed)
3. Right-click -> Send to Repeater -> edit pass, resend, watch the response
4. Send to Intruder -> mark 'pass' -> load rockyou.txt -> Sniper -> run
5. Sort by response LENGTH -> the outlier row is the successful login
6. Send to Comparer -> diff success vs failure -> confirm the tell
7. Log the finding with the exact request (the documentation habit)
```

This one flow — Proxy -> Repeater -> Intruder -> Comparer — is the spine of nearly every web test you will run this term.

PART D

The OWASP Top 10 (2021)

The industry's map of web risk — and the syllabus for this web block

WHAT IT IS

The OWASP Top 10 is an awareness document

A community-built, periodically-updated ranking of the most critical web application security risks — a shared vocabulary, not a complete checklist.

- Built by OWASP (owasp.org) from real-world data across many organisations; the current stable edition used here is 2021.
- It is a starting point and a common language — 'we found an A01' means the same thing to every professional.
- For exhaustive coverage a tester uses OWASP WSTG (from the methodology topic); the Top 10 is the headline risk map above it.
- It is revised over time: OWASP published a 2025 update, but this course teaches the well-established 2021 list and its numbering.

Analogy: *A public-health 'top causes' list — it tells you where to focus first, not every disease that exists.*

REFERENCE TABLE

OWASP Top 10 — 2021

The ten categories, in rank order — the map for the whole web block (delivered across this Part and the two that follow):

ID	CATEGORY	COVERED IN
A01	Broken Access Control	Part 1 (today)
A02	Cryptographic Failures	Part 3
A03	Injection (SQLi, command, XSS)	Part 2 / 3
A04	Insecure Design	Part 3
A05	Security Misconfiguration	Part 3
A06	Vulnerable & Outdated Components	Part 3
A07	Identification & Auth Failures	Part 1 (today)
A08	Software & Data Integrity Failures	Part 3
A09	Security Logging & Monitoring Fail.	Part 3 (brief)
A10	Server-Side Request Forgery (SSRF)	Part 2

Source: owasp.org/Top10/2021. Today covers the bookends — A01 (ranked #1) and A07. Note XSS sits under A03 Injection in the 2021 taxonomy.

WHERE IT'S HEADING

OWASP Top 10 — 2021 vs 2025

The standard is periodically re-ranked from fresh breach data. The 2025 edition (the 8th, now final) moved things — awareness only; this course is examined on 2021:

2021		2025	CHANGE
A01 Broken Access Control	->	A01 Broken Access Control	= #1 (SSRF added)
A02 Cryptographic Failures	->	A04 Cryptographic Failures	down 2
A03 Injection	->	A05 Injection	down 2
A04 Insecure Design	->	A06 Insecure Design	down 2
A05 Security Misconfiguration	->	A02 Security Misconfiguration	UP 3
A06 Vulnerable/Outdated Comp.	->	(folded into A03 Supply Chain)	merged
A07 Ident. & Auth Failures	->	A07 Authentication Failures	= renamed
A08 Software/Data Integrity	->	A08 Software/Data Integrity	=
A09 Logging & Monitoring	->	A09 Logging & Alerting Failures	= renamed
A10 SSRF	->	(folded into A01 Access Control)	merged
(new in 2025)	->	A03 Software Supply Chain Fail.	NEW
(new in 2025)	->	A10 Mishandling of Exceptions	NEW

Verified against owasp.org/Top10/2025. Two new categories (Supply Chain, Mishandling of Exceptions); SSRF and Vulnerable Components absorbed; Misconfiguration rose to #2. Broken Access Control stays #1 — this Part's content only grew in importance.

How the Top 10 splits across this web block

The three parts of the web block divide the list by theme so each builds on the last.

- Part 1 (today): the access layer — A01 Broken Access Control and A07 Identification & Authentication Failures.
- Part 2: the injection family — A03 Injection built brick-by-brick (SQLi), command injection, XXE, and A10 SSRF.
- Part 3: client-side and logic — XSS and CSRF, A04 Insecure Design, A05 Misconfiguration, A06 Components, plus a brief look at request smuggling.
- A01 jumped from #5 (2017) to #1 (2021) and stayed #1 in the 2025 revision — access control has been the top web risk for years, which is why the block opens here.

Analogy: *Three surgeons splitting one long operation by region — access first, then the injection tract, then the surface and the logic.*

PART E

A01 — Broken Access Control

The #1 web risk: the server checks WHO you are, then forgets to check WHAT you may reach

TWO WORDS TO SEPARATE

Authentication vs authorisation

Two different questions the app must answer in order. Access control is the second one, and it is the one that breaks.

- Authentication (authn): 'who are you?' — proven by a password, token or key. This is Part F's territory.
- Authorisation (authz): 'are you allowed to do this?' — checked on every single request, for every resource.
- Broken Access Control is an authorisation failure: the app knows exactly who you are and still lets you reach what you should not.
- The classic mistake: check authentication once at login, then never re-check authorisation on the individual request.

Analogy: *Authentication is showing your passport at the airport gate; authorisation is whether your visa actually lets you into that country's restricted zone.*

Three questions every request must survive

For each request the server must confirm identity, resource ownership, and permission — a miss on any one is Broken Access Control.

- Who is making this request? (a valid, authenticated identity — Part F)
- What exact resource or function are they asking for? (this order, this file, this admin action)
- Is THIS identity allowed THIS resource, right now? (the check that is so often skipped)
- The bug is almost always the third question being unasked — the server trusts that the UI would never have shown the option.

Analogy: *A bouncer who checks your name, sees the guest list, but never checks whether your wristband is the VIP colour — the third check is the one that gets skipped.*

TWO DIRECTIONS

Vertical vs horizontal access control

Access can break upward to higher privilege, or sideways to another user at the same level — test both.

- Vertical: a normal user reaches an admin-only function — deleting users, viewing all orders, changing roles.
- Horizontal: a user reaches another user's data at the same privilege level — someone else's invoice, profile or message.
- Horizontal is the one juniors forget: it is not about becoming admin, it is about being any other user.
- Both reduce to the same failure — the server never re-checked the third question for this specific identity.

Analogy: *Vertical is a guest slipping into the manager's office; horizontal is a guest walking into another guest's room using their own key.*

THE FLAGSHIP BUG

IDOR — Insecure Direct Object Reference

The app puts an object's identifier in a request the client controls, and then trusts it without checking ownership.

- A request like GET /api/orders?id=1042 returns order 1042 — and changing 1042 to 1041 returns someone else's order.
- The reference (the id) is 'direct' because it maps straight to a database row, and 'insecure' because no ownership check guards it.
- It is a horizontal access-control failure and the single most common access-control finding on the web.
- Sequential integer IDs make it trivial; predictable references (email, username, short codes) are almost as bad.

Analogy: *A hotel where the key for room 237 also opens 238, because the lock only checks the number you typed, not who you are.*

WORKED EXAMPLE

IDOR on the lab, step by step

Against OWASP Juice Shop (:3000) — accessing another user's basket by its id:

```
# 1. Log in as user A, open your basket, watch the request in Burp:
GET /rest/basket/1   Authorization: Bearer <A's JWT>   -> A's items (OK)

# 2. Send to Repeater. Change the basket id 1 -> 2, keep A's token:
GET /rest/basket/2   Authorization: Bearer <A's JWT>

# 3. Response returns basket 2 — a DIFFERENT user's items.
#    The server checked the token is valid (authn) but never checked
#    that basket 2 BELONGS to user A (authz). That gap is the IDOR.

# curl form of the tampered request:
$ curl -s http://localhost:3000/rest/basket/2 -H 'Authorization: Bearer <A_JWT>'
```

Same pattern on PortSwigger Academy's access-control labs (portswigger.net/web-security/access-control). The tell: valid session, someone else's data.

THE METHOD

How to hunt IDOR systematically

Two accounts, every object reference, change it to the other account's, and watch for data that is not yours.

- Register two test accounts (A and B) — the two-context setup is the backbone of all access-control testing.
- In every request, find the object references: numeric IDs, UUIDs, filenames, account numbers, order codes.
- As user A, replace each reference with B's value (or the next integer) and resend in Repeater; Intruder-Sniper to enumerate a range.
- Always include a same-account positive control: confirm A's own id still works, so a failure really means 'blocked', not 'broken request'.

Analogy: *Testing a lock by trying your own key in your neighbour's door — and first checking your key still opens your own, so a failure is meaningful.*

IN-CLASS EXERCISE · 6 min

Is this IDOR?

As user A (id 1001), you capture these three requests. Which are Broken Access Control, and what single test confirms each?

- GET /api/users/1001/profile returns A's profile. Changing 1001->1002 returns B's full profile.
- GET /api/invoice/8830.pdf returns A's invoice. Changing 8830->8831 returns 403 Forbidden.
- GET /api/messages?box=inbox returns A's messages. Adding &user=1002 returns B's messages.
- DELETE /api/users/1002 as user A returns 200 OK and removes user B.

Forced browsing to restricted functions

If an admin page is protected only by not being linked, then knowing its URL is enough to reach it.

- Security by obscurity: /admin, /admin/deleteUser, /api/internal/ exist but appear in no menu for a normal user.
- The Part B content-discovery pass (ffuf/gobuster) finds them; the access-control test is whether a low-privilege user can actually load them.
- A 403 means the check works; a 200 means the only thing protecting the function was the missing link — a serious finding.
- Test both the page and its actions: sometimes the admin page is hidden but the action endpoint it calls is not protected at all.

Analogy: *Trying door numbers that were left off the lobby directory — some of them are unlocked simply because nobody expected you to knock.*

WORKED EXAMPLE

Forced browsing — hidden admin function

Discovery found /admin returns 403 for a normal user. Test the function directly (lab targets only):

```
# Normal user hits the admin PAGE -> blocked:
$ curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/admin \
       -H 'Cookie: session=<low_priv_user>'      ->  403

# But the ACTION the page calls may not re-check the role:
$ curl -s -X POST http://localhost:8080/api/user/delete \
       -H 'Cookie: session=<low_priv_user>' -d 'id=1002'   ->  200 OK

# 200 on the action = missing function-level access control.
# The page was guarded; the endpoint behind it was not.
```

This is 'Missing Function Level Access Control' — the page-level check exists but the underlying API action trusts the request. Test endpoints, not just pages.

The rest of the A01 family

Beyond IDOR and forced browsing, Broken Access Control has several cousins worth naming now and testing all term.

- Mass assignment: an update endpoint accepts extra fields the UI never sends — add "role":"admin" or "isVerified":true (found via Part B param discovery).
- Privilege stored client-side: a role=user cookie or a JWT claim the client can edit — change it and become admin (attacked in Part F).
- CORS misconfiguration: an over-permissive cross-origin policy that lets another site read authenticated responses (deep in Part 3).
- Path traversal: ../../ in a file parameter to read files outside the intended directory (deep in Part 2).

Analogy: *The same failure wearing different coats — every one is the server trusting a value the client was free to set.*

THE FIX

Fixing Broken Access Control

Deny by default, enforce every decision on the server, and check ownership on every request — never rely on hiding.

- Deny by default: every resource is forbidden unless a rule explicitly grants this identity access.
- Enforce server-side, centrally: one authorisation layer every request passes through, never a per-page check the developer might forget.
- Check ownership, not just login: 'is this record owned by this user?' on every object access — the fix for IDOR.
- Never use unpredictable IDs as the control: UUIDs raise the bar but are not authorisation; and log access-control failures (A09) to detect probing.

Analogy: *The kitchen re-checks every ticket against the table that ordered it — the dining room is never trusted to enforce who gets which dish.*

PART F

A07 — Auth failures

Breaking the 'who are you?' — weak credentials, broken sessions, and forgeable tokens

What A07 actually covers

Every failure in confirming identity and then maintaining it — from the login form to the session token to the reset flow.

- Credentials: weak-password acceptance, credential stuffing, and brute-force with no rate limit or lockout.
- Identity leaks: username enumeration that tells an attacker which accounts are real.
- Sessions: predictable, stealable, or never-expiring session tokens that let an attacker ride an identity.
- Recovery & tokens: broken password-reset flows and forgeable authentication tokens such as weak JWTs.

Analogy: *A building where the ID check at the door is weak AND the visitor badge never expires — two separate failures, one intruder inside.*

Weak credentials and credential stuffing

The strongest login logic is undone by a password of '123456' or one reused from a public breach.

- Weak-password acceptance: the app permits 'password', 'admin', or the username itself — testable directly at signup.
- Credential stuffing: attackers replay username/password pairs leaked from other breaches, betting on reuse.
- Password spraying: one common password tried across many accounts, to stay under per-account lockout thresholds.
- A login test at the auth layer uses hydra; deep offline password cracking (hashes with hashcat) belongs to the network-security topic.

Analogy: *A bank vault with a world-class door and '1234' as the combination — the door strength is irrelevant.*

IDENTITY LEAK

Username enumeration

If the app reveals which usernames exist, an attacker has already won half the login — they now only need the password.

- Tells: a login that says 'unknown user' vs 'wrong password', a signup that says 'email already registered', a reset that behaves differently for real accounts.
- Timing side-channel: a real account takes longer (the password is actually hashed and checked) than a non-existent one.
- The impact multiplier: a confirmed user list turns spraying and stuffing from guesswork into targeted attacks.
- OWASP WSTG-IDNT-04 (account enumeration) is the checklist reference; Burp Intruder automates the probe.

Analogy: *A receptionist who says 'no one by that name works here' for strangers but 'wrong password' for real staff — now you know exactly who is real.*

WORKED EXAMPLE

Detecting username enumeration

Same request, two usernames — the response delta reveals which account exists:

```
# Non-existent user:
POST /login   user=notreal&pass=x
-> 200  'No account with that username'   (fast, ~40ms)

# Real user, wrong password:
POST /login   user=admin&pass=x
-> 200  'Incorrect password'              (slow, ~180ms)

# Two different messages AND a timing gap both leak account validity.
# Burp Intruder (Sniper) over a username list -> sort by length/time.
```

The fix is a single generic message ('invalid username or password') and constant-time handling — so real and fake accounts are indistinguishable.

NO BRAKES

Missing rate-limiting and lockout

A login that accepts unlimited guesses turns any weak password into a certainty, given time.

- Without a lockout or rate limit, Intruder or hydra can try thousands of passwords against one account.
- A 6-digit OTP or PIN with no rate limit is only a million values — minutes to hours at machine speed.
- Defences to test for: account lockout after N failures, exponential backoff, CAPTCHA, and IP/velocity throttling.
- Real engagements cap request rate in the RoE — a de-facto brute-force can itself cause an outage.

Analogy: *A PIN pad that lets you keep guessing forever — 10,000 combinations is an afternoon, not a wall.*

The session token is the crown jewel

After login, the whole of your identity is one token — steal it and you are the user, no password required.

- HTTP is stateless (Part A), so after authentication the app issues a session token the client returns on every request.
- That token is a bearer credential: possession equals identity — this is why cookie theft via XSS (Part 3) is so severe.
- It must be unpredictable (high-entropy, not a sequential or guessable value), transmitted only over TLS, and invalidated on logout.
- If the token can be predicted, stolen, or reused after logout, authentication is broken no matter how strong the password was.

Analogy: *A festival wristband: after the ticket check nobody looks at your face again — steal the band and you walk in freely.*

How sessions are broken

Three classic failures: guess the token, fixate it before login, or reuse it after logout.

- Prediction: low-entropy or sequential tokens can be guessed or brute-forced with Intruder — test the randomness.
- Session fixation: the app keeps the same token across the login boundary, so a token planted before login is valid after it.
- No invalidation: the token still works after logout, after password change, or long past any sensible expiry.
- Missing cookie flags (next slide) expose the token to theft over HTTP or via JavaScript.

Analogy: *Session fixation is handing someone a wristband before they enter, then walking in on their arm once they are admitted.*

REFERENCE TABLE

Session cookie flags a tester checks

Read the Set-Cookie header (Part A) and confirm each protective flag is present:

FLAG	WHAT IT DOES	MISSING =
HttpOnly	Hides cookie from JS	XSS can steal it (Part 3)
Secure	HTTPS-only transmission	Leaks over plain HTTP
SameSite	Limits cross-site sending	CSRF easier (Part 3)
Expires/Max	Bounds the lifetime	Token lives forever
Path/Domain	Scopes where it is sent	Over-broad exposure

A good session cookie: Set-Cookie: session=<random>; HttpOnly; Secure; SameSite=Lax. Missing HttpOnly is what makes XSS escalate to full account takeover.

JWT — anatomy of a JSON Web Token

A self-contained token of three Base64url parts: anyone can read it, and the signature is the only thing stopping tampering.

- Structure: header.payload.signature — three Base64url segments separated by dots.
- Header: the algorithm (e.g. HS256 or RS256). Payload: the claims — user id, role, expiry — in plain, readable JSON.
- Signature: a MAC or digital signature over header+payload, the ONLY thing that makes the claims trustworthy.
- Critical point: the payload is encoded, not encrypted — Decoder (Part C) or jwt.io reveals every claim instantly.

Analogy: *A tamper-evident parcel: anyone can read the label through the window; only the seal proves who actually packed it.*

WORKED EXAMPLE

A JWT, decoded

One token, split on the dots and Base64url-decoded (in Burp Decoder or at jwt.io):

```
eyJhbGciOiJIUzI1NiJ9  .  eyJ1c2VyIjoiYW1hbCIsInJvbGUiOiJ1c2VyIn0  .  <sig>
        header                        payload                    signature

header  -> { "alg": "HS256", "typ": "JWT" }
payload -> { "user": "amal", "role": "user", "exp": 1789050000 }
signature -> HMAC-SHA256( base64(header) + '.' + base64(payload), secret )

# The role claim says 'user'. If the signature check is weak or skipped,
# an attacker changes it to 'admin' and re-signs -> instant privilege gain.
```

Decoding needs no secret — that is by design. Forging a valid token needs the secret (or a flaw that removes the need for one, next slide).

Three ways JWTs are broken

Every JWT attack targets the signature check — remove it, guess its key, or confuse which key is used.

- alg:none — set the header algorithm to 'none' and strip the signature; a server that honours it accepts any forged payload.
- Weak HMAC secret — HS256 signed with a guessable secret ('secret', 'password') can be cracked offline, then any token is forgeable.
- Algorithm confusion (RS256 -> HS256) — trick a server into verifying an RS256 token with HS256, using the public key as the HMAC secret.
- All three end the same way: change role to admin, produce a token the server accepts — authentication and A01 fall together.

Analogy: *alg:none is a parcel where the sender writes 'no seal required' on the box — and a careless courier just believes it.*

TYPE THESE TONIGHT

Testing JWTs with jwt_tool

jwt_tool (github.com/ticarpi/jwt_tool) automates the three attacks — against lab tokens only:

```
# Decode and inspect a token:
$ python3 jwt_tool.py <JWT>

# Tamper mode — edit claims interactively (e.g. role -> admin):
$ python3 jwt_tool.py <JWT> -T

# Exploit: try the alg:none bypass:
$ python3 jwt_tool.py <JWT> -X a

# Crack a weak HMAC secret with a wordlist, then forge at will:
$ python3 jwt_tool.py <JWT> -C -d /usr/share/wordlists/rockyou.txt
```

Verified against the jwt_tool docs: -T tamper, -X a for the none-algorithm attack, -C -d to crack the signing key. Burp's 'JWT Editor' extension does the same inside Burp.

Password reset and MFA weaknesses

The recovery path is authentication's back door — and multi-factor auth only helps if it is actually enforced.

- Reset-token flaws: predictable or non-expiring tokens, or a reset link built from a spoofable Host header that sends the token to an attacker's domain.
- Reset that leaks: the new-password link exposed in a referer header or returned in the response body.
- MFA not enforced: the second factor is skipped by navigating straight to the post-login URL, or replayed because the code has no rate limit.
- These are pointers today; multi-factor bypass and account-takeover chains are drilled deeper in the exploitation block.

Analogy: *A locksmith who mails the replacement key to whatever return address is written on the request form — the front door is irrelevant now.*

THE FIX

Fixing Identification & Authentication Failures

Strong credentials, enforced MFA, hardened sessions, and tokens whose signatures are actually verified.

- Credentials: block breached and weak passwords, add rate-limiting and lockout, and support MFA — enforced on every sensitive action, not just login.
- Anti-enumeration: identical generic messages and constant-time responses so real and fake accounts are indistinguishable.
- Sessions: high-entropy tokens, HttpOnly + Secure + SameSite cookies, rotation on login, and invalidation on logout and password change.
- Tokens: verify the signature with a fixed server-side algorithm, reject alg:none, use a strong secret or key, and keep expiry short.

Analogy: *A door that resists picking, a badge that expires and re-checks the face, and a signature the guard genuinely verifies — defence in depth on identity.*

PART G

Mitigations & takeaways

One principle under both categories — and where the block goes next

THE UNIFYING DEFENCE

One principle under both A01 and A07

Broken access control and broken authentication are two faces of the same failure: the server trusting the client to enforce the rules.

- A01 trusts the client's claim about WHAT it may access; A07 trusts the client's claim about WHO it is — both are the Part A trust boundary, broken.
- The universal fix is server-side enforcement: identity, permission, and session validity are decided where the attacker cannot reach — the server.
- Deny by default, verify every request, and treat everything from the browser as hostile until the server has re-checked it.
- Every remaining week of this block is another instance of the same boundary — injection, XSS and logic flaws all begin with trusted client input.

Analogy: *The kitchen re-checks every ticket against the table — the dining room is never trusted to decide who gets served.*

IN-CLASS EXERCISE · 8 min

Triage the request

You are testing an authorised app as a low-privilege user. For each captured request, name the category (A01 or A07), the specific bug, and the one test that confirms it.

- GET /api/reports/774 returns your report; 775 returns the finance team's report.
- The session cookie is Set-Cookie: sid=1001 — and yours was 1000 a minute ago.
- The login says 'no such user' for random names but 'wrong password' for 'admin'.
- Your JWT payload reads role:user; the header reads alg:HS256 with an obvious secret.

THE PART 1 MESSAGE

Every web vulnerability is the server trusting something the client controls. Broken access control trusts what you may reach; broken authentication trusts who you claim to be. Move the fight to the raw HTTP with Burp, and enforce every decision on the server.

KEY TAKEAWAYS

The five things to remember from Part 1

1. **The client is hostile territory** — everything the browser sends is forgeable — security must live server-side
2. **Burp is the workflow** — Proxy to capture, Repeater to tamper, Intruder to automate, Comparer to read the delta
3. **A01 is an authorisation bug** — IDOR and forced browsing — the server knows who you are and forgets to check what you may reach
4. **A07 breaks identity** — weak credentials, enumeration, stealable sessions, and forgeable JWTs
5. **Two accounts + positive control** — the method that turns an access-control hunch into a confirmed finding

NEXT IN THIS BLOCK

Part 2 — Injection (A03), built brick by brick

Same trust boundary, sharper edge: when the server treats client input as code instead of data.

- SQL injection built step by step: detect, UNION, blind, time-based, stacked, then the fix — plus sqlmap to automate it.
- Command injection, XXE, and SSRF chaining out to cloud metadata; path traversal and insecure file upload.
- The through-line is unchanged: injection is the server trusting client input enough to execute it — Part A's boundary, weaponised.
- Cross-reference for later: prompt injection is 'the new injection' against AI features — the full AI block is Topic 9, covered separately.

Analogy: *Today the server trusted WHO and WHAT; in Part 2 it trusts client input so completely it runs it as its own instructions.*

References & Further Reading

- Prescribed text: D. Stuttard & M. Pinto, The Web Application Hacker's Handbook, 2nd ed., Wiley — Ch. 4 (Mapping), Ch. 8 (Access Controls), Ch. 6-7 (Authentication & Sessions)
- OWASP Top 10:2021 (owasp.org/Top10/2021) — A01 Broken Access Control, A07 Identification & Authentication Failures
- OWASP Web Security Testing Guide v4.2 — Authentication (WSTG-ATHN), Authorization (WSTG-ATHZ), Session Management (WSTG-SESS), Identity (WSTG-IDNT)
- Burp Suite (portswigger.net/burp); OWASP ZAP (zaproxy.org)
- Tools: ffuf (github.com/ffuf/ffuf), gobuster, feroxbuster, whatweb, Arjun, jwt_tool (github.com/ticarpi/jwt_tool)
- Wordlists: SecLists (github.com/danielmiessler/SecLists); token inspection at jwt.io
- Practice (authorised only): OWASP Juice Shop (:3000), DVWA (:8080), PortSwigger Web Security Academy — Access control & Authentication (portswigger.net/web-security)
- Note: OWASP published a 2025 revision; this course uses the stable 2021 edition and its A01-A10 numbering